SIC Transmetro — Para qué, por qué y qué soluciona

Justificación del sistema y de sus decisiones de arquitectura · Proyecto final de Aseguramiento de la Calidad del Software · 2 de octubre de 2026

1. El problema que existe hoy

La información del Transmetro está repartida entre cada municipalidad y cada operador. Nadie tiene la foto completa de las 7 líneas, las 106 estaciones y las 210 unidades al mismo tiempo. Las reglas de operación existen en papel y en la costumbre, pero se verifican a mano.

Problema actual	Qué provoca
Catálogos en archivos separados por municipalidad	El mismo dato existe en varias versiones y ninguna es la oficial
Asignación de unidades a líneas registrada en hojas de cálculo	Una unidad puede quedar contada en dos líneas, o en ninguna
Control del rango de unidades por observación	La falta de unidades se detecta cuando el usuario ya está esperando
Parqueos asignados verbalmente	Unidades que terminan el día sin espacio asignado
Cobertura de guardias en listas impresas	Accesos descubiertos que nadie nota hasta que pasa algo
Conteo de pasajeros sin registro	El sobrecupo no deja rastro y no se puede reclamar
Regla de espera de 5 minutos a criterio del operador	Se cumple cuando alguien se acuerda
Recaudo conciliado días después	Los faltantes se descubren tarde y ya no se pueden rastrear
Sin registro de quién cambió qué	Ante un error, no hay a quién preguntar ni cómo reconstruir

El sistema existe para cerrar esa brecha: una sola base de datos como fuente oficial, y las reglas verificadas por la máquina en lugar de por la memoria de una persona.

2. Para qué sirve el sistema

Para qué	Quién lo usa
Tener un catálogo único y oficial de líneas, estaciones, accesos y unidades	Administrador municipal y general
Saber en todo momento cuántas unidades opera cada línea y si está dentro de su rango	Administrador municipal y general
Garantizar que ninguna unidad esté asignada a dos líneas ni se quede sin parqueo	Administrador municipal
Llevar el expediente completo de pilotos y la cobertura de guardias por acceso	Administrador municipal, piloto, guardia
Registrar la ocupación real de cada unidad y detectar el sobrecupo	Operador de estación y piloto
Hacer cumplir la regla de espera de 5 minutos	Operador de estación
Conocer el estado del equipo de cómputo y la conectividad de cada estación	Operador de estación
Conciliar el recaudo por tarjeta el mismo día	Operador de estación
Obtener reportes consolidados de toda la red, exportables	Administrador general
Saber quién hizo cada cambio, cuándo y qué valores tocó	Auditoría y administración

3. Las decisiones de arquitectura, una por una

Cada decisión se explica igual: para qué sirve, por qué se eligió frente a la alternativa y qué problema concreto soluciona.

3.1 Una sola base de datos relacional centralizada

Para qué. Que exista un único lugar donde consultar cualquier dato de la red, sin importar qué municipalidad lo capturó.

Por qué. La alternativa era mantener una base por municipalidad y consolidarlas cada cierto tiempo. Eso significa que durante el intervalo entre consolidaciones nadie tiene el dato correcto, y que las diferencias hay que resolverlas a mano. Se eligió PostgreSQL porque es de código abierto, exigido por el proyecto, y porque es el único motor libre que trae las tres herramientas que este sistema necesita: restricciones de exclusión, columnas generadas y triggers maduros.

Qué soluciona. Elimina las versiones paralelas del mismo dato. La municipalidad sigue siendo dueña de su información — es una columna en cada tabla — pero la información vive en un solo sitio.

3.2 Las reglas de negocio dentro de la base de datos

Para qué. Que una regla se cumpla siempre, sin importar por dónde entren los datos.

Por qué. Lo habitual es programar las reglas en el backend. El problema es que entonces la regla solo se cumple si el dato entró por ahí: una carga masiva, un script de corrección o una consulta manual en pgAdmin la saltan sin avisar. Al ponerla en la base, la validación es parte del dato, no del programa que lo escribe.

Qué soluciona. Que una regla escrita en el documento de requerimientos se cumpla de verdad en producción, y no solo en el camino feliz de la aplicación.

Regla	Mecanismo	Qué impide

RF-04 Un bus en una sola línea activa	**EXCLUDE USING gist sobre unidad y periodo**	**Que la misma unidad se cuente en dos líneas, aunque dos personas la asignen al mismo tiempo**
RF-06 Ninguna unidad sin parqueo	**Dos restricciones EXCLUDE**	**Que dos unidades ocupen el mismo espacio, o que una unidad aparezca en dos predios**
RF-09 Sobrecupo	**Columnas generadas al 150 % de la capacidad**	**Que el sobrecupo dependa de que alguien saque la cuenta**
RF-10 Espera de 5 minutos	**CHECK sobre la fila del viaje**	**Que se despache con menos del 25 % de ocupación antes de tiempo**
RF-05 Rango de unidades	**Trigger que abre y cierra la alerta**	**Que una línea opere fuera de su rango sin que nadie se entere**
RF-08 Cobertura de guardias	**Trigger contra el mínimo del acceso**	**Que un acceso quede descubierto sin aviso**

3.3 Arquitectura en tres capas con API REST

Para qué. Que la lógica viva en un solo lugar y que todos los clientes — la web de hoy, una aplicación móvil mañana, un tablero de Power BI — obtengan el mismo resultado.

Por qué. La alternativa era una aplicación que hablara directo con la base. Eso obliga a repetir las validaciones en cada pantalla y a abrir la base a cada equipo de la red, que es un riesgo de seguridad. Con una API de por medio, la base solo acepta conexiones del servidor de aplicación.

Qué soluciona. Evita que cada nueva pantalla reimplemente las reglas, y permite cambiar la interfaz sin tocar la lógica. También cumple el requisito no funcional de capas desacopladas.

3.4 Interfaz web responsiva por componentes

Para qué. Que el mismo sistema se use desde la computadora de la estación y desde un teléfono en el andén.

Por qué. El operador y el guardia trabajan de pie. Una aplicación de escritorio los obligaría a volver a la caseta para registrar lo que acaban de ver, y el registro tardío es registro inventado. Se eligió React por ser componentes reutilizables y porque es el estándar con más documentación y soporte, lo que importa en un sistema que otra persona va a mantener.

Qué soluciona. Que la ocupación de una unidad se registre en el momento y en el lugar donde ocurre.

3.5 La matriz de permisos como tabla, no como código

Para qué. Que cada uno de los cinco roles vea y modifique exactamente lo que le corresponde.

Por qué. Programar los permisos con condiciones dentro del código significa que cada ajuste es un cambio de programa, una prueba y un despliegue. Al guardarlos en una tabla, ajustar un permiso es actualizar una fila.

Qué soluciona. Que un piloto no pueda ver el expediente de otro, que un operador no modifique el catálogo de líneas y que un administrador municipal no toque datos de otra municipalidad. Y que corregir un permiso mal asignado tome un minuto, no un despliegue.

3.6 Ámbito de datos separado del nivel de permiso

Para qué. Distinguir *qué puede hacer* un usuario de *sobre cuáles filas* puede hacerlo.

Por qué. Los dos administradores municipales tienen el mismo nivel de permiso, pero no deben ver los mismos datos. Mezclar las dos ideas obliga a crear un rol por municipalidad, y la lista crece sin control cada vez que entra un municipio nuevo.

Qué soluciona. Que el sistema escale a más municipios sin rediseño: agregar uno es insertar una fila, no crear roles nuevos.

3.7 Alertas generadas por la base, no por una revisión periódica

Para qué. Que las situaciones anómalas se detecten en el instante en que ocurren.

Por qué. La alternativa era un proceso que revisara cada cierto tiempo. Eso deja una ventana ciega entre revisiones y consume recursos preguntando por cosas que no cambiaron. Un trigger reacciona al cambio mismo.

Qué soluciona. Que la alerta por una línea bajo su mínimo exista en el mismo segundo en que la unidad entra a mantenimiento, y que se cierre sola cuando la condición se normaliza, sin que nadie tenga que acordarse de bajarla.

3.8 Los datos derivados no se almacenan

Para qué. Que la longitud de una línea, el porcentaje de ocupación y la diferencia de conciliación siempre coincidan con los datos que los componen.

Por qué. Un total guardado es un total que alguien debe recordar actualizar. Tarde o temprano se olvida y el sistema empieza a mentir con seguridad.

Qué soluciona. Que no existan reportes que se contradigan entre sí.

3.9 Bitácora de auditoría con el usuario real

Para qué. Reconstruir qué pasó, cuándo y por decisión de quién.

Por qué. Lo habitual es que la aplicación escriba en la bitácora cuando el programador se acuerda, y que la base registre al usuario técnico de la conexión, que es el mismo para todos. Aquí el identificador de la persona viaja hasta la base en cada transacción, y un trigger genérico cubre 14 tablas críticas.

Qué soluciona. Que ante un dato incorrecto se sepa quién lo cambió y qué valor tenía antes, sin depender de que alguien recuerde. Es información pública sujeta al Decreto 57-2008 y la trazabilidad es parte de la obligación.

3.10 Base administrada en la nube

Para qué. Tener respaldo automático, réplica y recuperación sin un equipo de infraestructura.

Por qué. Un servidor propio exige a alguien que verifique los respaldos cada día. En la práctica eso se descubre fallando el día que se necesita restaurar.

Qué soluciona. El requisito de respaldo y plan de recuperación ante desastres, y la disponibilidad de 99 % en horario operativo.

4. Qué soluciona el sistema, requerimiento por requerimiento

RF	Para qué sirve	Qué problema manual soluciona
RF-01	Catálogo de líneas y estaciones	Elimina las listas paralelas de cada municipalidad
RF-02	Distancia entre estaciones y longitud de línea	Evita el cálculo a mano y los totales desactualizados
RF-03	Inventario de accesos	Da la base para saber cuántos guardias se necesitan
RF-04	Un bus en una sola línea activa	Impide la doble asignación y el conteo inflado de flota
RF-05	Alerta por rango de unidades	Avisa antes de que el usuario note la falta de servicio
RF-06	Asignación de parqueos	Ninguna unidad termina el día sin espacio
RF-07	Expediente de pilotos	Reemplaza folders en papel y licencias vencidas sin detectar
RF-08	Cobertura de guardias por acceso	Detecta el acceso descubierto antes del turno
RF-09	Ocupación y sobrecupo	Deja evidencia del sobrecupo, que hoy no existe
RF-10	Regla de espera de 5 minutos	La vuelve obligatoria en lugar de voluntaria
RF-11	Estado de equipo y conectividad	Permite planificar mantenimiento en vez de reaccionar a la falla
RF-12	Conciliación de pagos	Detecta el descuadre el mismo día, cuando aún se puede rastrear
RF-13	Asociación a municipalidad	Define con claridad quién responde por cada estación y línea
RF-14	Reportes consolidados	Sustituye la recolección manual de datos de cada operador

RF-15	**Usuarios y roles**	**Cada quien ve lo suyo, sin cuentas compartidas**
RF-16	**Bitácora de auditoría**	**Da trazabilidad ante reclamos y auditorías**

5. Qué está resuelto y qué falta

Estado	Alcance
Implementado y probado	**Modelo de datos completo (32 tablas), reglas RF-04, RF-05, RF-06, RF-08, RF-09, RF-10 y RF-16 dentro de la base, seguridad y matriz de permisos (RF-15), y los módulos de líneas, estaciones, accesos, flota y parqueos (RF-01 a RF-06)**
Modelado, pendiente de construir	**Personal (RF-07, RF-08), monitoreo (RF-09, RF-10), técnica (RF-11), pagos (RF-12) y reportería (RF-14). Las tablas y las reglas ya existen; falta la interfaz y los endpoints**
Verificación realizada	**El script se ejecutó completo en PostgreSQL 18.3, dos veces seguidas, sin errores. Las reglas se probaron con datos válidos e inválidos: las inválidas fueron rechazadas por la base**

Lo que este diseño compra, en una frase: convierte reglas que hoy dependen de que una persona se acuerde en restricciones que la base de datos no deja violar, y deja rastro de todo cambio.